Labo 5: Geavanceerde Computerarchitectuur

Gust Brouckaert
Tim Depickere

1 december 2024

1 Introductie

In dit labo onderzochten we **asynchrone operatie** binnen CUDA-programmering. De focus lag op het begrijpen van hoe kernel-functies elkaar kunnen aanroepen en hoe CPU en GPU parallel kunnen werken zonder blokkering. De opdracht omvatte het uitvoeren van reductie-operaties (som, product, minimum, en maximum) en het analyseren van de prestatieverschillen tussen synchrone en asynchrone uitvoering.

2 Opdrachtbeschrijving en Implementatie

De opdracht bestond uit twee delen:

- **Deel 1:** Reductie-operaties uitvoeren (som, product, minimum, en maximum) op vier onafhankelijke arrays synchroon
- **Deel 2:** Deel 1 opnieuw uitvoeren asynchroon.

De kern van onze implementatie is de volgende kernel-functie, die generiek is voor alle reductie-operaties:

Listing 1: CUDA-kernel voor somreductie

```
__global__ void sumDetectionGPUReduction(int *array, int size) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    for (int i = 1; i < size; i *= 2) {
        int index = 2 * i * idx;
        if (index < size) {
            array[index] = array[index] + array[index + i];
        }
        __syncthreads();
    }
}
```

Elke reductie-operatie volgt een vergelijkbare structuur, waarbij specifieke logica wordt toegepast (bijvoorbeeld optellen, vermenigvuldigen of het minimum bepalen). We gebruikten CUDA-streams voor asynchrone verwerking en synchroniseerden per stream om overlappende verwerking mogelijk te maken.

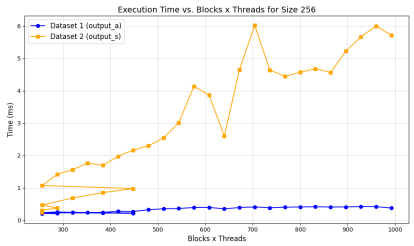
De belangrijkste stappen in de implementatie zijn:

1. Initialiseren van een dataset op de CPU.
2. Overbrengen van data naar de GPU (`cudaMemcpy` voor synchroon, `cudaMemcpyAsync` voor asynchroon).
3. Aanroepen van reductie-kernels voor elke operatie, synchroon of asynchroon.
4. Resultaten terughalen naar de CPU (`cudaMemcpy` voor synchroon, `cudaMemcpyAsync` voor asynchroon).
5. Meten van de uitvoertijd met `cudaEventElapsedTime`.

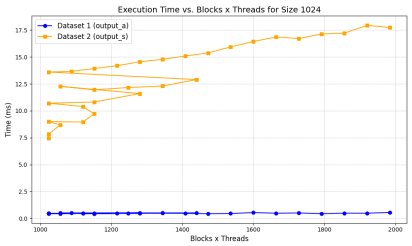
3 Prestaties en Analyse

We vergeleken de uitvoertijd van de synchrone en asynchrone workflow, evenals het effect van dynamisch parallelisme. De resultaten zijn samengevat in onderstaande grafieken.

3.1 Resultaten



(a) Vergelijking van synchrone en asynchrone uitvoeringstijd bij arraysize van 256.



(b) Vergelijking van synchrone en asynchrone uitvoeringstijd bij arraysize van 1024.

Figure 1: Prestatie-analyse van verschillende uitvoeringsstrategieën.

- **Asynchrone uitvoering:** Zoals zichtbaar in de bovenstaande figuur, resulteerde asynchrone verwerking in aanzienlijke prestatiewinst, vooral bij grotere datasets. Voor een arraysize van 1024 was het verschil in uitvoeringstijd met de synchrone methode gemiddeld een factor 35. Bij kleinere arrays, zoals 256, was het verschil minder uitgesproken, met een factor 10. Dit illustreert hoe asynchrone verwerking efficiënter omgaat met grotere workloads.

- **Schaalbaarheid:** De prestatiewinst bij asynchrone verwerking neemt toe naarmate de dataset groter wordt. Dit komt doordat de GPU beter in staat is om overlappende operaties uit te voeren, terwijl de CPU doorgaat met andere taken. Synchrone uitvoering blijft echter eenvoudiger te implementeren en is voldoende voor kleine datasets.

4 Conclusie

Dit labo toonde aan dat asynchrone verwerking binnen CUDA aanzienlijke prestatiewinst kan opleveren, vooral bij grotere datasets. Door data-overdracht en kernel-executie te overlappen, wordt de GPU optimaal benut en worden uitvoeringstijden drastisch verlaagd. Synchrone verwerking is eenvoudiger, maar minder geschikt voor grotere workloads.